Infrastructure as Code Foundations

Part 2 — From IaC design to automated delivery

SEAtS | 549471

University of Southampton

Intended Learning Outcomes

By the end of this session you will be able to:

1. Decompose an IaC system into modules with clear, versioned interfaces, and manage providers and third-party IP.
2. Relate the service delivery platform and framework to how IaC artifacts are built and deployed.
3. Use formal specification (schemas, typed variables, policy-as-code) to make infrastructure enforceable.
4. Compare the three ways to write IaC (JSON, DSL, general-purpose language) and explain Pulumi's hybrid model.
5. Trace the Pulumi project/stack workflow from `pulumi new` through `up` to `destroy`.
6. Plan a testing strategy and a CI/CD and GitOps pipeline for infrastructure changes.
7. Distinguish provisioning, deployment, orchestration and configuration management.
8. Identify common IaC risks and anti-patterns and propose mitigations.

© University of Southampton

Agenda

1. Designing IaC systems: modules and interfaces.
2. Service delivery and formal specification.
3. Writing IaC: JSON, DSL, Vagrant examples.
4. Pulumi: hybrid model and workflow.
5. Testing, CI/CD and GitOps.
6. Provisioning, orchestration, CM.
7. Risks, anti-patterns, and summary.

© University of Southampton

Designing IaC Systems

Decomposing into Blocks

Example module folders:

- `/networking`
- `/identity`
- `/compute`
- `/database`
- `/monitoring`

Identifying Interfaces

Interfaces via:

- Input variables.
- Output variables.
- Remote state references.

Interfaces must be clear, versioned, documented, and stable.

Designing APIs & Protocols

IaC interacts with:

- Cloud provider APIs.
- Terraform provider schemas.
- Kubernetes manifests/CRDs.

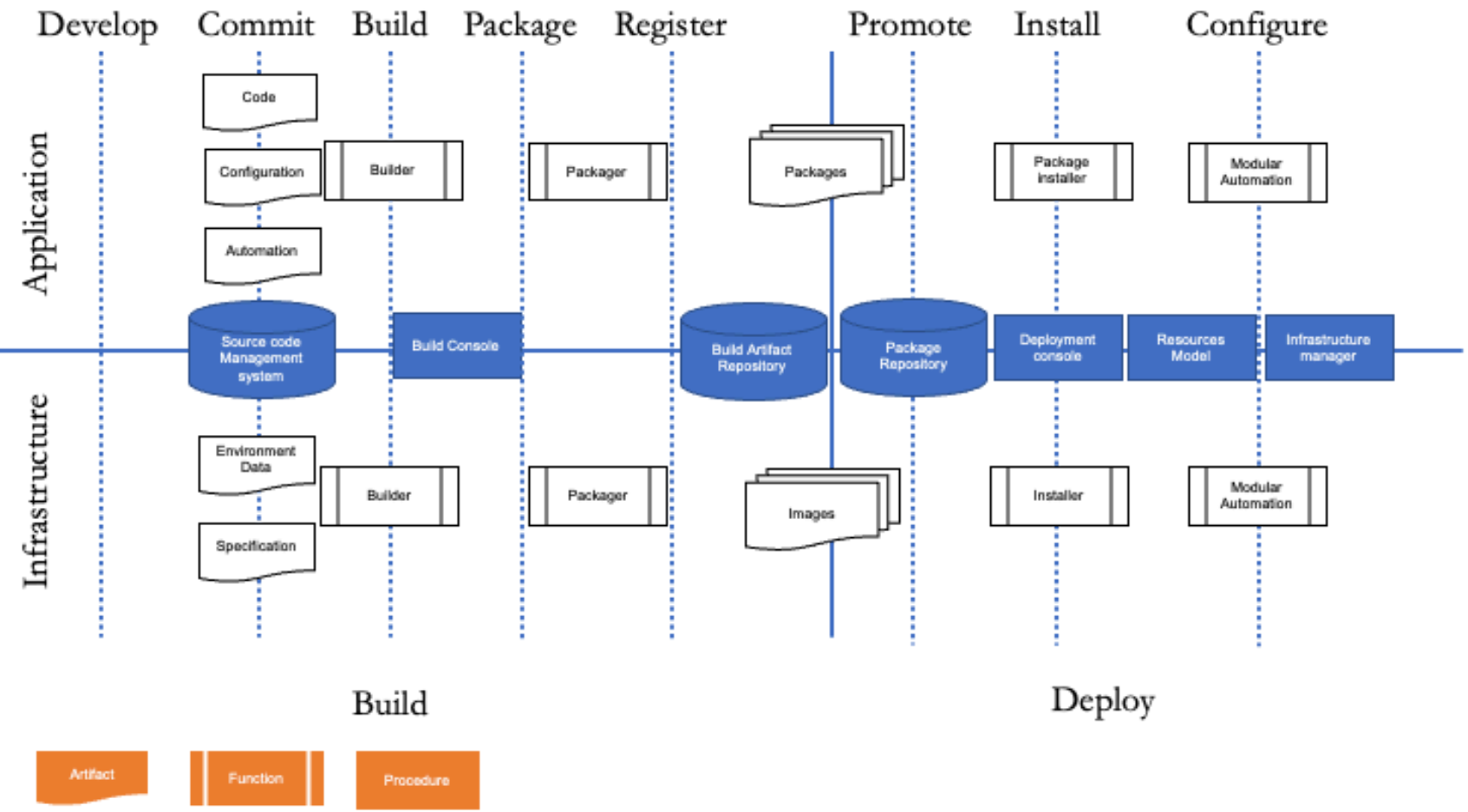
Integrating Providers & IP

- Cloud providers (AWS, Azure, GCP).
- Third-party providers (Cloudflare, Datadog).
- Internal libraries reused across teams.

Managing Third-Party IP

- Version pinning and lifecycle management.
- Licensing and compliance considerations.
- Security scanning of modules/providers.

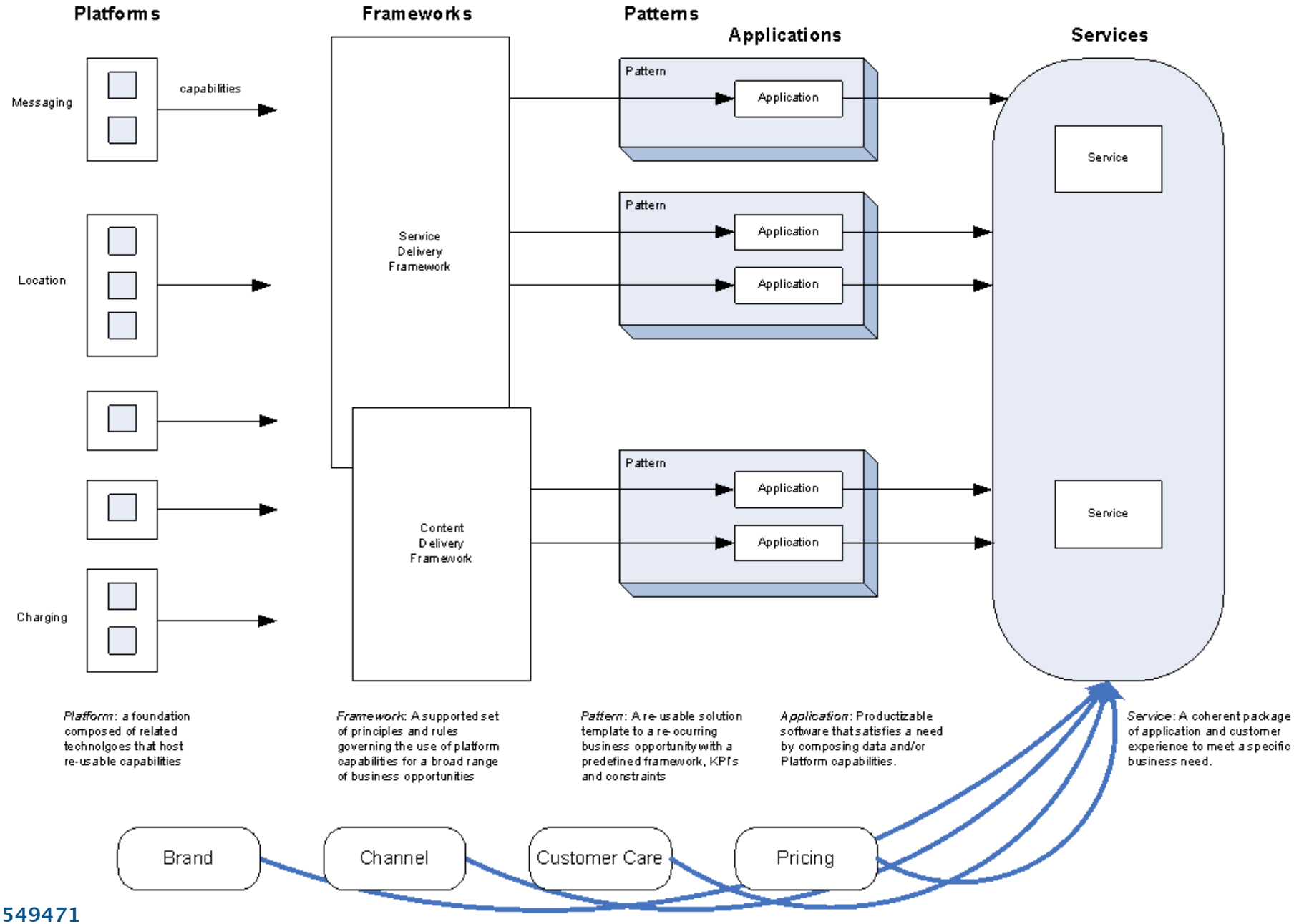
© University of Southampton



Service Delivery Platform

- **Artifact:** Versioned, tested, deployed units.
- **Procedures (Imperative):** Commands to produce desired state (e.g., Bash).
- **Functions (Declarative):** Define desired state; tooling conforms system (e.g., SQL, makefiles).

© University of Southampton



© University of Southampton

Formal Specification in IaC

- JSON/YAML schemas (CloudFormation, Kubernetes).
- Terraform variable typing and module interfaces.
- Policy-as-Code (OPA/Rego) for compliance.
- Cloud policies (AWS Config, Azure Policy) to enforce encryption, logging, segmentation.

Formalisation supports reproducibility and enforcement.

© University of Southampton

Three Ways toWrite IaC

Method	Description
JSON	Resources (EC2 instance, security group, …) listed as structured data for the provider to consume
Domain-specific language (DSL)	A custom syntax built for one tool/provider — looks code-like, isn't a general-purpose language
General-purpose programming language	A real language (Pulumi supports TypeScript, JavaScript, Python, Go, .NET) used to describe infrastructure

The general-purpose-language approach brings **loops, conditionals, classes, and packages** to infrastructure definitions — e.g. a `for` loop that creates one instance per availability zone — plus normal IDEs, linters, and test frameworks.

© University of Southampton

YAML Examples

```
Vagrant.configure("2") do |config|
  config.vm.box = "peru/ubuntu-18.04-desktop-amd64"
  config.vm.box_version = "2.3"
  config.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "1024"
    vb.name = "ubuntudesktopscyther"
  end
end
```

```
servers = YAML.load_file('servers.yaml')
Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|
  servers.each do |server|
    config.vm.define server["name"] do |srv|
      srv.vm.box = server["box"]
      srv.vm.network "private_network", ip: server["ip"]
      srv.vm.provider :virtualbox do |vb|
        vb.name = server["name"]
        vb.memory = server["ram"]
      end
    end
  end
end
```


© University of Southampton

Pulumi : hybrid?

Pulumi is a hybrid, not a straightforward "imperative" tool: its *state model* is declarative — you describe desired infrastructure and `pulumi up` diffs it against reality, exactly like Terraform — but its *authoring language* is a real general-purpose language (Python), which is imperative. Two separate axes: **what gets reconciled** vs **how you express it**.

Declarative state reconciliation scales better than hand-written procedural steps for large multi-component systems — true whether you write that declaration in HCL (Terraform) or in Python (Pulumi).

© University of Southampton

From Theory to Pulumi: The Workflow

A **project** is a single program that declares desired infrastructure; a **stack** is one deployed instance of that project (e.g. `dev` vs `prod`) — same program, different config (region, parameters), deployed independently. Neither term appears in the theory chapters above — this is the vocabulary that was missing.

sequenceDiagram actor Dev participant CLI as Pulumi CLI participant Cloud as AWS Dev->>CLI: pulumi new <template> CLI-->>Dev: scaffolds project + "dev" stack Dev->>CLI: pulumi config set aws:region ... Dev->>CLI: pulumi up CLI->>CLI: diff desired state (code) vs actual state CLI-->>Dev: preview of resources to create/update Dev->>CLI: confirm (yes) CLI->>Cloud: create/update resources Cloud-->>CLI: resulting resource state CLI-->>Dev: outputs (e.g. bucket name, URL) Dev->>CLI: pulumi stack output <name> CLI-->>Dev: exported value Note over Dev,CLI: edit main.py, re-run "pulumi up" — only the minimal diff is applied Dev->>CLI: pulumi destroy CLI->>Cloud: tear down all stack resources Dev->>CLI: pulumi stack rm <name>

© University of Southampton

Testing IaC Systems

Testing types:

1. Static testing (linting: tflint, checkov; policy-as-code).
2. Unit tests (Terratest, Molecule).
3. Integration tests (ephemeral deployments).
4. Acceptance tests (VPC routing, IAM permissions).
5. Security tests (CSPM, IaC scanning).

© University of Southampton

Build Systems & CI

Standard Pipeline

1. Validate (`terraform validate`).
2. Lint (tflint, checkov).
3. Plan (static analysis, diffs).
4. Approvals (human-in-loop).
5. Apply (automated, controlled).
6. Post-deploy tests (Terratest, smoke tests).

Continuous Deployment (GitOps)

- ArgoCD / Flux for declarative sync.
- Atlantis, Spacelift, Terraform Cloud for collaboration.
- Automated rollbacks based on policy drivers.

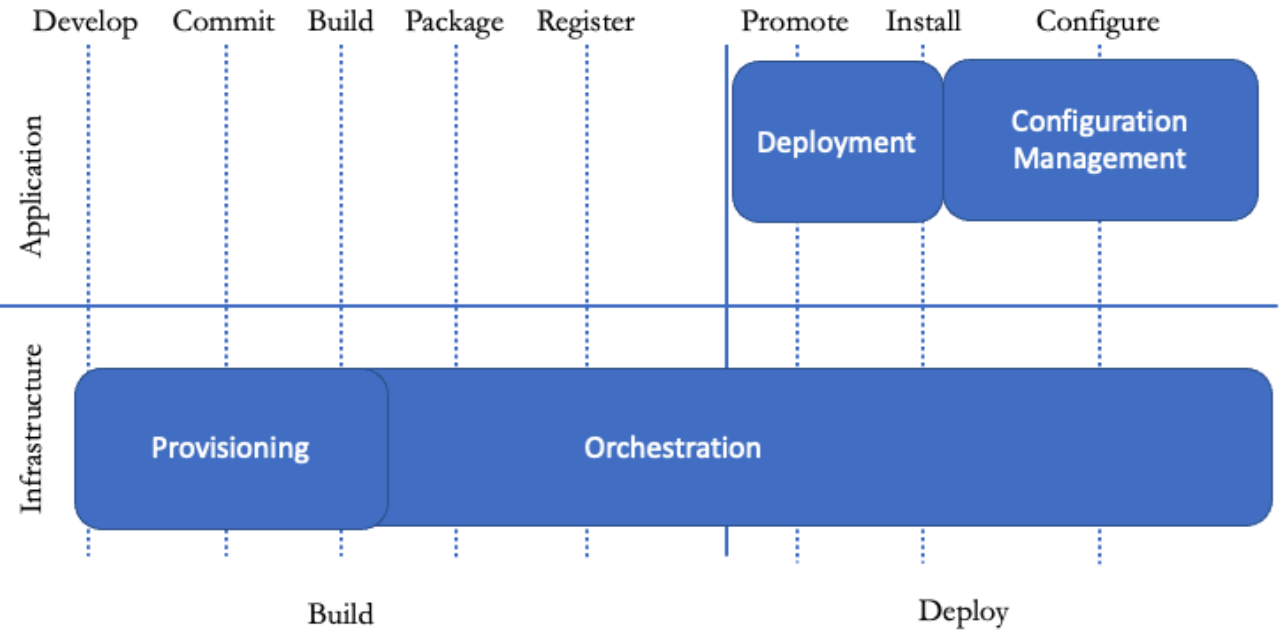
Managing Development

Methodologies: Agile, DevOps, GitOps, infrastructure release management.

- Multi-team decomposition, module backlogs, environment versioning.
- Branch protection, review bots (tfsec), semantic versioning.
- Automated changelogs and release notes.

© University of Southampton

Deployment / Orchestration / CM



- **Provisioning:** Prepare server (hardware, OS, services, network).
- **Deployment:** Automated app deployment & upgrades.
- **Orchestration:** Coordinated operations across systems.
- **Configuration Management:** Maintain app dependencies.

© University of Southampton

Risks & Anti-Patterns

- Configuration drift.
- Hard-coded secrets.
- Unversioned remote state.
- No testing or code reviews.
- Monolithic Terraform files, parallel changes without locking.
- Black-box modules with no documentation.

Mitigation: code reviews, state locking, policy-as-code, decomposition.

© University of Southampton

Summary

- Decompose infrastructure into modules whose interfaces (inputs, outputs, remote state) are clear, versioned and stable; pin, licence-check and scan third-party providers.
- Formal specification (schemas, typed variables, policy-as-code) makes infrastructure reproducible and enforceable.
- IaC can be written as JSON, a DSL, or a general-purpose language; Pulumi's *state model* is declarative even though its *authoring language* is imperative Python.
- `pulumi new` → `up` → `destroy` is the same provisioning loop regardless of what you deploy; stacks run the same code with different config.
- Test IaC at every level, and ship it through CI/CD (validate → lint → plan → approve → apply → test) and GitOps.
- Mitigate anti-patterns (drift, secrets, unlocked state, monoliths) through reviews, locking, policy-as-code and decomposition.

© University of Southampton